GARL-ACO: A Graph-aware Reinforcement Learning Framework for Adaptive Parameter Control in Ant Colony Optimization

Van-Tan Bui*✉, Anh-Chien Khuat*, Phuong-Thao Tran*
*Faculty of Information Technology, University of Economics - Technology for Industries
† VNU University of Engineering and Technology
Email: bvtan@uneti.edu.vn

✉ Corresponding author.

***Abstract*—Ant Colony Optimization (ACO) is an effective metaheuristic for routing and combinatorial optimization; however, its performance is highly sensitive to control parameters that are typically predefined and kept fixed throughout the search process. Such static parameter settings often degrade solution quality in dynamic routing problems with time-varying edge costs. This paper proposes GARL-ACO (Graph-Aware Reinforcement Learning for Ant Clony Optimization), a novel framework for online parameter control in Ant Colony Optimization for the Dynamic Traveling Salesman Problem (DTSP). GARL-ACO employs a Deep Deterministic Policy Gradient (DDPG) agent to continuously adjust three key ACO parameters—the pheromone influence factor α, heuristic influence factor β, and pheromone evaporation rate ρ—according to the current optimization state and graph dynamics. To enable effective decision-making, a 34-dimensional graph-aware state representation is constructed by integrating 18 graph and environmental features with 16 optimization- and parameter-related features. In addition, a dynamic routing environment with periodically perturbed edge costs is designed to emulate time-varying transportation networks. The proposed framework is evaluated on five TSPLIB benchmark instances and compared with both fixed-parameter ACO and a Q-learning-based adaptive parameter controller. Experimental results demonstrate that GARL-ACO consistently achieves the best solution quality across all evaluated instances while exhibiting superior robustness under dynamic graph perturbations. In particular, on the large-scale `u1060` instance, GARL-ACO reduces the optimality gap from 737.08% with fixed ACO and 43.82% with Q-learning to only 27.55%. Moreover, the learned policy achieves immediate recovery on four of the five benchmark instances and generalizes effectively to the previously unseen `pcb1173` instance, where it attains an optimality gap of 25.04%. These results demonstrate that integrating graph-aware state representations with continuous reinforcement learning provides an effective and scalable solution for adaptive parameter control in dynamic combinatorial optimization.**

***Keywords*-Dynamic Traveling Salesman Problem, Ant Colony Optimization, Deep Reinforcement Learning, Graph-aware Learning, Adaptive Parameter Control, Route Optimization.**

I. Introduction

Many real-world routing and network optimization problems, such as intelligent transportation, urban logistics, autonomous vehicle routing, and delivery scheduling, can be formulated as graph-based combinatorial optimization problems. The Traveling Salesman Problem (TSP) is one of the most extensively studied problems in this class and has long served as a representative benchmark for evaluating combinatorial optimization algorithms. In practical transportation and logistics systems, however, travel conditions are rarely static. Traffic congestion, accidents, road restrictions, and other environmental factors may cause edge costs to vary over time, resulting in the Dynamic Traveling Salesman Problem (DTSP). Unlike its static counterpart, the DTSP requires an optimization algorithm not only to identify high-quality routes but also to rapidly adapt its search behavior when the underlying cost structure changes [1], [2].

Ant Colony Optimization (ACO) is a population-based metaheuristic inspired by the collective foraging behavior of ant colonies and has been successfully applied to a wide range of routing and combinatorial optimization problems. Classical variants, including Ant System (AS), Ant Colony System (ACS), and Max–Min Ant System (MMAS), exploit indirect communication through artificial pheromone trails to iteratively construct and reinforce promising solutions [3], [4], [2]. This distributed search mechanism provides a natural balance between exploration and exploitation and makes ACO particularly suitable for graph-based optimization. Nevertheless, its performance is strongly influenced by several control parameters, particularly the pheromone influence factor

α, heuristic influence factor β, and pheromone evaporation rate ρ. In conventional ACO, these parameters are typically selected through empirical tuning and remain fixed during optimization. A parameter configuration that performs well at one stage of the search or under one graph condition may become inappropriate after the search dynamics or edge costs change, potentially causing premature convergence, excessive exploration, or slow adaptation to newly emerging high-quality routes [5], [2].

Adaptive parameter control provides a promising mechanism for reducing this dependence on manually predefined configurations. Rather than using a fixed parameter set throughout the optimization process, an adaptive controller can modify the search behavior according to the current optimization condition. Reinforcement Learning (RL) is particularly attractive for this purpose because parameter control can be formulated as a sequential decision-making problem: the optimization process provides a state, the controller selects an appropriate parameter configuration as an action, and the subsequent search improvement provides feedback for learning. In contrast to handcrafted adjustment rules, an RL-based controller can learn parameter-control policies directly through interaction with the optimization environment [6]. Deep Reinforcement Learning (DRL) further extends this capability to complex state representations and has demonstrated considerable potential for learning decision policies in combinatorial optimization and graph-related problems [7], [8], [9].

A key challenge, however, lies in determining what information should be provided to the learning agent. Search statistics such as solution improvement, convergence behavior, stagnation, and pheromone distribution describe the internal state of ACO but do not fully characterize changes in the optimization landscape. This limitation becomes particularly important for the DTSP, where variations in edge costs directly alter the search space perceived by the optimizer. Graph-level properties, edge-weight distributions, local connectivity patterns, and the magnitude of environmental perturbations can therefore provide complementary information for adaptive parameter control. A controller that jointly observes the optimization process and the underlying graph can potentially distinguish search stagnation caused by the algorithm from performance degradation caused by environmental changes.

Another important consideration is the representation of the parameter-control action space. The three major ACO parameters α, β, and ρ are inherently continuous. Discretizing them into a limited number of predefined configurations simplifies the learning problem but restricts the controller to coarse parameter transitions. Such discretization may be particularly limiting in dynamic environments, where different magnitudes and patterns of graph perturbation may require fine-grained adjustments. Deep Deterministic Policy Gradient (DDPG) provides an actor–critic framework specifically designed for continuous action spaces [10]. This property makes DDPG suitable for learning an online parameter-control policy that directly maps the observed state to continuous values of the ACO control parameters.

Motivated by these observations, this paper proposes GARL-ACO, a framework for continuous online parameter control in ACO under dynamic graph conditions. GARL-ACO integrates the ACO search process with a DDPG agent that dynamically controls α, β, and ρ during optimization. Instead of representing the environment solely through conventional search statistics, GARL-ACO employs a 34-dimensional graph-aware state representation comprising 18 graph and environmental features and 16 search- and parameter-related features. The graph-aware component characterizes graph size, edge-weight distribution, connectivity-related information, local neighborhood structure, and environmental perturbations, whereas the search component captures solution quality, convergence, recent improvement, stagnation, pheromone statistics, current parameter values, and optimization progress. Based on this joint representation, the DDPG actor generates continuous parameter values that are immediately applied to the subsequent ACO search cycle.

GARL-ACO is evaluated in DTSP environments constructed from multiple TSPLIB instances, where a subset of edge costs is periodically perturbed to emulate time-varying routing conditions. In addition to conventional fixed-parameter ACO, a Q-learning-based adaptive controller is implemented as a learning-based baseline. This baseline uses discretized search states and a finite set of parameter combinations, enabling an empirical comparison between discrete parameter selection and graph-aware continuous control. The evaluation considers

solution quality, optimality gap, variability across independent runs, and recovery behavior after environmental perturbations. The learned GARL-ACO policy is also evaluated on an unseen TSPLIB instance to examine its ability to transfer beyond the instances used during training.

The main contributions of this work are summarized as follows:

- We propose GARL-ACO, which formulates online ACO parameter adaptation in dynamic routing environments as a continuous reinforcement learning problem. Its DDPG-based controller jointly adjusts the pheromone influence factor α, heuristic influence factor β, and pheromone evaporation rate ρ during optimization.
- We introduce a 34-dimensional graph-aware state representation that combines 18 graph and environmental features with 16 search- and parameter-related features. This representation enables GARL-ACO to capture both changes in the underlying optimization environment and the internal search dynamics of ACO.
- We establish a closed-loop interaction mechanism among the dynamic graph environment, ACO search process, graph-aware state representation, and DDPG controller, allowing parameter configurations to be continuously adapted in response to environmental perturbations and search progress.
- We conduct comparative experiments on multiple TSPLIB instances against fixed-parameter ACO and a discrete Q-learning-based adaptive controller. GARL-ACO achieves the lowest mean optimality gap on all evaluated instances and substantially improves performance on large-scale problems. In particular, on the `u1060` instance, it reduces the mean gap from 737.08% for fixed ACO and 43.82% for Q-learning to 27.55%.
- We further assess cross-instance generalization using the unseen `pcb1173` instance. Without including this instance during training, GARL-ACO achieves a mean gap of 25.04%, compared with 37.61% for Q-learning and 69.02% for fixed ACO, providing empirical evidence that the learned parameter-control policy can transfer to a previously unseen problem instance.

The remainder of this paper is organized as follows. Section II reviews related work on ACO parameter control, reinforcement learning for combinatorial optimization, and learning-assisted adaptive optimization. Section III presents GARL-ACO, including the dynamic graph environment, graph-aware state representation, continuous parameter controller, reward function, and training procedure. Section IV describes the experimental setup and analyzes the comparative results. Finally, Section V concludes the paper and discusses limitations and directions for future research.

Motivated by these observations, this work proposes GARL-ACO, a graph-aware continuous parameter-control framework for adaptive ACO in dynamic routing environments. Unlike existing approaches that rely solely on search-level indicators, GARL-ACO employs a 34-dimensional state representation that combines 18 graph and environmental features with 16 search- and parameter-related features. Based on this graph-aware state, a DDPG actor jointly generates continuous values for α, β, and ρ. GARL-ACO integrates the learned controller with ACO in a closed-loop optimization process, allowing parameter configurations to be continuously adapted as both the search dynamics and graph environment evolve. The experimental evaluation compares GARL-ACO with fixed-parameter ACO and a discrete Q-learning-based adaptive controller, and further assesses its cross-instance generalization on an unseen TSPLIB benchmark.

II. Proposed Method

In this section, we present GARL-ACO, a novel framework that integrates graph-aware state representation with Deep Deterministic Policy Gradient (DDPG) to dynamically control the key parameters of Ant Colony Optimization in dynamic routing environments.

A. Problem Formulation

This work considers the Dynamic Traveling Salesman Problem (DTSP), in which the travel cost between cities varies during the optimization process while the graph topology remains unchanged. Let

$$G_t = (V, E, W_t) \tag{1}$$

denote the dynamic weighted graph at optimization step t, where $V = \{v_1, v_2, \ldots, v_N\}$ is the set of vertices, E is the set of edges, and $W_t = \{w_{ij}(t)\}$ represents the time-varying edge-weight matrix.

Given a Hamiltonian tour

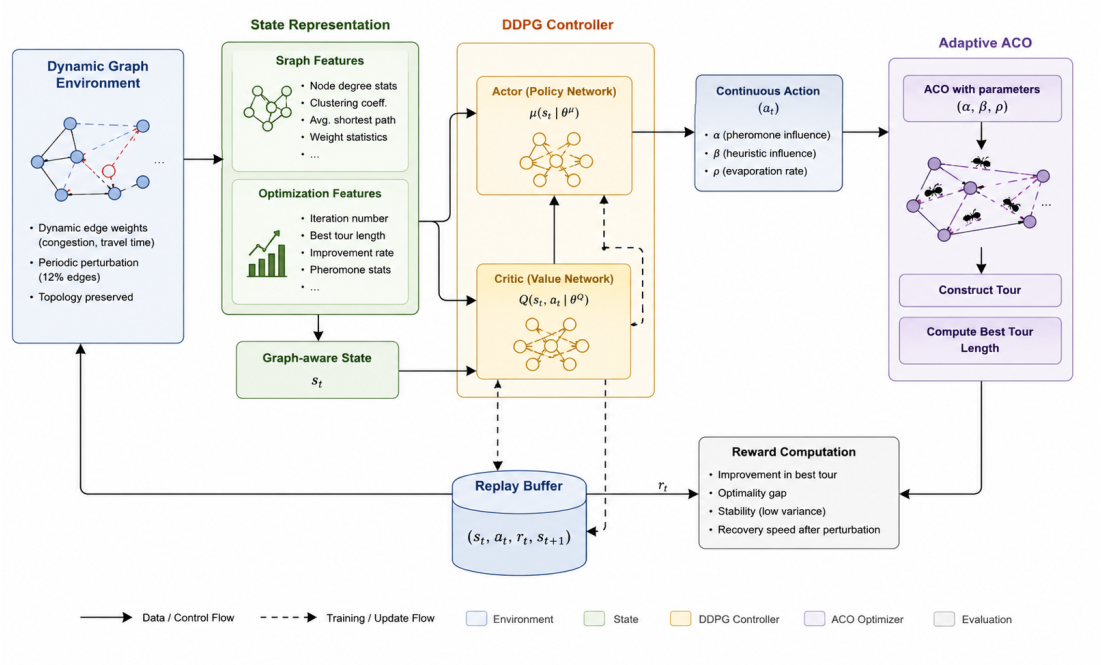


Figure 1: Overall architecture of the proposed GARL-ACO framework. The framework integrates graph-aware state representation with a DDPG-based controller to continuously adjust the ACO parameters (α, β, ρ) according to both graph characteristics and search dynamics.

$$\pi = (v_1, v_2, \ldots, v_N, v_1), \tag{2}$$

its travel cost under graph state G_t is

$$L(\pi, t) = \sum_{k=1}^{N} w_{\pi_k, \pi_{k+1}}(t), \tag{3}$$

where $\pi_{N+1} = \pi_1$.

The objective of DTSP is to continuously identify high-quality tours despite the evolution of edge weights over time,

$$\pi^*(t) = \arg\min_{\pi} L(\pi, t). \tag{4}$$

Unlike the conventional static TSP, the optimal solution of DTSP may change whenever the underlying edge costs are updated. Consequently, an optimization algorithm must continuously adapt its search strategy rather than converge to a single static optimum.

In this work, Ant Colony Optimization (ACO) is employed as the underlying optimizer. During each iteration, ants construct candidate tours according to the current pheromone distribution and heuristic information. The search behavior of ACO is primarily governed by three continuous control parameters:

- the pheromone influence factor α;
- the heuristic influence factor β;
- the pheromone evaporation rate ρ.

These parameters jointly determine the balance between exploration and exploitation throughout the optimization process. In conventional ACO, they are selected empirically before optimization and remain fixed during the entire search. Such a static parameter configuration is often ineffective for DTSP because the search landscape evolves as the graph changes. A parameter setting that is suitable at one stage of the search may become suboptimal after environmental perturbations, leading to slower adaptation, premature convergence, or degraded solution quality.

Instead of treating parameter selection as an offline hyperparameter tuning problem, we formulate adaptive parameter control as a sequential decision-making problem. At each control cycle, an agent observes the current optimization state together with the characteristics of the dynamic graph and determines the parameter configuration

$$a_t = (\alpha_t, \beta_t, \rho_t). \tag{5}$$

The objective is therefore to learn a policy

$$\pi_\theta : s_t \rightarrow (\alpha_t, \beta_t, \rho_t), \tag{6}$$

which maximizes the long-term optimization performance by continuously adapting the ACO parameters to both the optimization dynamics and the evolving graph environment.

B. Overall Framework

Figure 1 illustrates the overall architecture of the proposed Adaptive Graph-aware Reinforcement Learning framework. The proposed framework tightly integrates four major components, namely the dynamic graph environment, the ACO optimization module, the graph-aware state builder, and the Deep Deterministic Policy Gradient (DDPG) controller, into a closed-loop optimization process.

At the beginning of each control cycle, the dynamic environment updates the edge weights according to the current traffic perturbation model. The updated graph is then passed to the ACO optimization module, which performs several search iterations using the current parameter configuration (α, β, ρ). During this process, the optimizer not only generates candidate solutions but also produces various search statistics that characterize the current optimization behavior.

Subsequently, the graph-aware state builder constructs a unified state representation by combining two complementary sources of information. The first source consists of graph- and environment-related descriptors, including graph structural properties, edge-weight distributions, connectivity characteristics, and perturbation statistics. These features characterize the current condition of the optimization environment. The second source contains optimization-related descriptors generated by the ACO search, such as solution quality, convergence behavior, pheromone statistics, stagnation information, and the current parameter values. Together, these two groups of features provide a comprehensive description of both the external environment and the internal search dynamics.

The resulting graph-aware state vector is fed into the DDPG controller. Unlike discrete reinforcement learning methods that select one configuration from a finite set of predefined parameter combinations, the actor network directly predicts continuous values of α, β, and ρ. These parameters are immediately applied to the subsequent ACO search cycle, thereby enabling smooth and fine-grained regulation of the exploration–exploitation balance.

After the updated parameters have been applied, the optimization improvement achieved during the current control cycle is evaluated to produce a reward signal. The transition consisting of the current state, selected action, received reward, and subsequent state is stored in the replay buffer and used to update the actor and critic networks. Consequently, parameter adaptation becomes a continuous closed-loop learning process in which each decision is guided by both the current graph condition and the observed optimization performance.

Compared with conventional adaptive ACO approaches that rely on manually designed parameter schedules or heuristic adjustment rules, the proposed framework possesses three distinctive characteristics. First, it explicitly incorporates graph structural information into the reinforcement learning state representation, enabling the controller to perceive environmental changes rather than relying solely on optimization statistics. Second, it performs joint continuous control of the three key ACO parameters instead of selecting from a discrete parameter set. Finally, the entire optimization process is organized as an end-to-end feedback loop in which graph evolution, ACO search, state construction, reinforcement learning, and parameter adaptation are tightly coupled. This design enables the proposed framework to continuously adjust its search strategy in response to both optimization dynamics and environmental perturbations.

C. Adaptive ACO Optimization Module

The proposed framework employs Ant Colony Optimization (ACO) as the underlying optimizer for solving the DTSP. Instead of redesigning the original search mechanism, this work focuses on improving its adaptability through online parameter control. Specifically, the reinforcement learning agent continuously adjusts the three key ACO parameters, namely the pheromone influence factor α, the heuristic influence factor β, and the pheromone evaporation rate ρ, while the remaining optimization procedure follows the standard ACO paradigm.

Given the current graph state, each ant incrementally constructs a feasible tour according to the probabilistic transition rule

$$P_{ij}^{k}(t) = \frac{\tau_{ij}(t)^{\alpha}\eta_{ij}^{\beta}}{\sum_{l\in\mathcal{N}_i^k}\tau_{il}(t)^{\alpha}\eta_{il}^{\beta}}, \tag{7}$$

where $\tau_{ij}(t)$ denotes the pheromone intensity on edge (i,j), $\eta_{ij} = 1/d_{ij}$ is the heuristic visibility determined by the reciprocal of the edge cost, and $\mathcal{N}_i^k$ represents the feasible neighborhood of ant k.

After all ants complete their tours, pheromone trails are updated according to

$$\tau_{ij}(t+1) = (1-\rho)\tau_{ij}(t) + \Delta\tau_{ij}(t), \tag{8}$$

where

$$\Delta\tau_{ij}(t) = \sum_{k=1}^{m}\Delta\tau_{ij}^{k}(t), \tag{9}$$

and

$$\Delta\tau_{ij}^{k}(t) = \begin{cases} \dfrac{Q}{L_k}, & (i,j)\in T_k, \\ 0, & \text{otherwise}, \end{cases} \tag{10}$$

where m is the number of ants, L_k denotes the tour length generated by ant k, and T_k represents its corresponding tour.

To further improve convergence stability, an elitist reinforcement strategy is adopted by additionally depositing pheromone on the globally best tour found so far. Consequently, high-quality routes remain competitive while obsolete pheromone information is gradually eliminated through evaporation.

Unlike conventional ACO implementations that simply output the best solution after each iteration, the proposed optimization module also serves as a search-state generator for reinforcement learning. During every control cycle, the optimizer continuously records optimization statistics that characterize the current search dynamics. These include solution quality, convergence behavior, recent improvement, stagnation level, pheromone distribution, and colony diversity. Such information forms an essential component of the graph-aware state representation presented in Section III-E.

It is worth emphasizing that the proposed framework does not modify the search mechanism of ACO itself. Instead, it continuously regulates the optimization behavior through adaptive online control of (α,β,ρ). Consequently, the exploration–exploitation balance is dynamically adjusted according to both the optimization progress and the evolving graph environment.

D. Dynamic Graph Environment

Real-world transportation networks rarely remain static throughout the optimization process. Traffic congestion, road maintenance, weather conditions, and unexpected incidents continuously alter travel costs, causing the optimal routing solution to evolve over time. To realistically evaluate the adaptability of the proposed framework, a dynamic routing environment is constructed by introducing time-varying perturbations into the edge weights of TSPLIB instances.

Unlike dynamic graph models that modify graph topology, the proposed environment preserves the vertex and edge sets while periodically updating edge costs. This setting accurately reflects many practical routing scenarios where the road network remains unchanged but travel times fluctuate.

Following the implementation adopted in this work, approximately 12% of graph edges are randomly selected as dynamic edges. Their weights evolve according to a sinusoidal perturbation model,

$$w_{ij}(t) = w_{ij}^{0}\left(1 + A\sin\left(\frac{2\pi t}{T} + \phi_{ij}\right)\right), \tag{11}$$

where w_{ij}^{0} is the original edge weight, A denotes the perturbation amplitude, T is the perturbation period, and ϕ_{ij} is a randomly initialized phase associated with edge (i,j).

This perturbation model possesses three desirable properties. First, edge costs evolve continuously rather than changing abruptly, thereby providing a realistic simulation of traffic fluctuations. Second, different random phases allow distinct edges to exhibit heterogeneous temporal behaviors, resulting in a more challenging optimization environment. Third, the periodic perturbation repeatedly alters the optimization landscape throughout the search, requiring the parameter controller to continuously adapt instead of converging to a single static parameter configuration.

After every environmental update, the dynamic graph environment provides two categories of information to the learning framework. The first is the updated edge-weight matrix used by the ACO

optimizer to construct new tours. The second consists of several environment descriptors, including perturbation intensity, edge-weight variation, traffic fluctuation statistics, and other graph-related measurements. These descriptors characterize the current graph condition and are subsequently incorporated into the graph-aware state representation.

The interaction between the dynamic environment and the adaptive optimizer forms a closed-loop optimization process. At each control cycle, environmental perturbations modify the routing landscape, ACO explores the updated graph using the current parameter configuration, and the reinforcement learning agent determines a new parameter setting based on both the observed graph condition and the resulting optimization behavior. This continuous interaction enables the proposed framework to maintain effective search performance under dynamically changing routing environments.

E. Graph-Aware State Representation

The effectiveness of reinforcement learning-based parameter control strongly depends on whether the state representation provides sufficient information for distinguishing different search and environmental conditions. A state constructed solely from objective values or convergence statistics can describe how ACO is currently performing, but it cannot explicitly indicate whether a performance change is caused by search stagnation or by a variation in the underlying edge-cost structure. This limitation is particularly important in the DTSP, where the optimization landscape evolves during the search.

In this work, a state is considered *graph-aware* when it jointly characterizes the current graph environment and the internal search dynamics of the optimizer. Accordingly, the state observed at control step t is defined as

$$\mathbf{s}_t = [\mathbf{g}_t \,\|\, \mathbf{h}_t] \in \mathbb{R}^{34}, \tag{12}$$

where $\mathbf{g}_t \in \mathbb{R}^{18}$ denotes the graph and environmental feature vector, $\mathbf{h}_t \in \mathbb{R}^{16}$ represents the ACO search and parameter feature vector, and $\|$ denotes vector concatenation. The two feature groups answer two complementary questions: $\mathbf{g}_t$ describes the current optimization environment, whereas $\mathbf{h}_t$ characterizes the behavior of the optimizer within that environment.

Table I: Graph and environmental features in the proposed state representation.

No.	Feature	Description
1	`num_nodes_norm`	Normalized number of vertices.
2	`avg_degree_norm`	Normalized average degree.
3	`max_degree_norm`	Normalized maximum degree.
4	`avg_weight_norm`	Scaled mean edge weight.
5	`weight_std_ratio`	Edge-weight standard deviation divided by its mean.
6	`weight_skewness`	Skewness of the edge-weight distribution.
7	`est_diameter_ratio`	Estimated graph diameter relative to mean edge weight.
8	`algebraic_connectivity_proxy`	Proxy for weighted-graph connectivity.
9–14	Perturbation features	Ratio, magnitude, trend, timing, and variability of dynamic edge-cost changes.
15	`knn_weight_ratio`	Mean k-NN distance relative to the global mean edge weight.
16	`congestion_cluster_ratio`	Local clustering intensity of perturbed or congested edges.
17	`weight_range_ratio`	Edge-weight range relative to its mean.
18	`iqr_ratio`	Interquartile range relative to the mean edge weight.

1) Graph and Environmental Features: The graph-aware component contains 18 features derived from the current weighted graph and its time-dependent perturbations. These features describe graph scale, degree characteristics, edge-weight distribution, connectivity, local neighborhood structure, and the magnitude and temporal behavior of environmental changes. The graph feature vector can be written as

$$\mathbf{g}_t = [g_t^{(1)}, g_t^{(2)}, \ldots, g_t^{(18)}]. \tag{13}$$

Table I summarizes these features. The perturbation feature group comprises six descriptors representing quantities such as the proportion of affected edges, the magnitude and trend of edge-cost changes, and the elapsed time since the most recent environmental variation.

The edge-distribution features indicate whether the cost landscape is homogeneous or highly dispersed. Connectivity and neighborhood descriptors provide information about the global and local structure of the graph, while the perturbation features explicitly quantify changes introduced by the dynamic environment. Their combination enables the controller to distinguish graphs that may have similar current objective values but substantially different structural and environmental conditions.

2) Search and Parameter Features: The second component of the state consists of 16 features

Table II: ACO search and parameter features.

Feature Group	Included Features
Solution quality	`norm_best_len`, `norm_iter_best_len`, `norm_iter_mean_len`.
Convergence	`conv_factor` (tour-length dispersion).
Recent improvement	`imp_5`, `imp_20` (short- and medium-term improvement).
Stagnation	`norm_stagnation` (normalized stagnation length).
Pheromone statistics	`tau_norm_mean`, `tau_norm_min`, `tau_norm_max`, `tau_norm_std`, `tau_norm_entropy`.
Current parameters	`norm_alpha`, `norm_beta`, `norm_rho`, `progress_ratio`.

extracted from the ACO optimization process:

$$\mathbf{h}_t = [h_t^{(1)}, h_t^{(2)}, \ldots, h_t^{(16)}]. \tag{14}$$

These features are organized into five functional groups, as summarized in Table II.

The solution-quality and improvement features indicate whether the current parameter configuration is producing effective search progress. The convergence and stagnation descriptors identify insufficient population diversity or a prolonged absence of improvement. The pheromone statistics characterize the concentration of search experience across the graph; in particular, low pheromone entropy may indicate that the colony is becoming overly concentrated on a limited set of edges. The current parameter values are included to allow the agent to model the effect of its previous actions, while the progress ratio differentiates early exploratory stages from later stages in which stronger exploitation may be desirable.

Features with different numerical scales are normalized before being supplied to the controller. For a feature x, min–max scaling is generally expressed as

$$\widetilde{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min} + \varepsilon}, \tag{15}$$

where ε is a small constant preventing division by zero. This normalization prevents large-magnitude variables from dominating neural network inputs and improves the stability of policy learning.

By combining the two feature groups, the proposed representation enables the controller to associate a search response with its underlying cause. For example, stagnation accompanied by limited environmental variation may suggest excessive exploitation, whereas a sudden deterioration associated with substantial edge-weight changes may require rapid evaporation of obsolete pheromone information. This distinction provides the basis for graph-aware adaptive parameter control.

F. Continuous Parameter Controller

The three ACO control parameters α, β, and ρ are inherently continuous and interact in regulating the exploration–exploitation behavior of the colony. Discretizing them into a finite number of configurations limits the granularity of adaptation and may produce abrupt parameter transitions. The proposed framework therefore employs Deep Deterministic Policy Gradient (DDPG), an off-policy actor–critic algorithm for continuous control [10].

1) Actor Network and Action Mapping: The actor network, denoted by $\mu(\mathbf{s}_t \mid \theta^\mu)$, maps the 34-dimensional state directly to a normalized three-dimensional action:

$$\widehat{\mathbf{a}}_t = \mu(\mathbf{s}_t \mid \theta^\mu) = [\widehat{a}_{\alpha,t}, \widehat{a}_{\beta,t}, \widehat{a}_{\rho,t}]. \tag{16}$$

The actor is implemented as a multilayer perceptron with two hidden transformations. Layer Normalization is applied after the hidden linear layers to stabilize feature distributions and gradient propagation. A small initialization range is used for the output-layer weights to prevent early saturation of the output activation and excessively large parameter changes at the beginning of training.

The normalized actor outputs are linearly mapped to the feasible intervals of the ACO parameters. For a parameter $p \in \{\alpha, \beta, \rho\}$, the mapping is

$$p_t = p_{\min} + \frac{\widehat{a}_{p,t} + 1}{2} \left(p_{\max} - p_{\min}\right), \tag{17}$$

assuming that the actor output is bounded within $[-1, 1]$. The resulting continuous action is

$$\mathbf{a}_t = [\alpha_t, \beta_t, \rho_t]. \tag{18}$$

This action is applied to the next ACO control cycle. Thus, the controller does not directly construct or modify tours; instead, it changes the behavior of ACO by regulating the relative influence of pheromone information, heuristic visibility, and pheromone evaporation.

During training, temporally correlated Ornstein–Uhlenbeck noise is added to the deterministic action:

$$\mathbf{a}_t^{\text{train}} = \text{clip}\left(\mu(\mathbf{s}_t \mid \theta^\mu) + \boldsymbol{\epsilon}_t\right), \tag{19}$$

where $\boldsymbol{\epsilon}_t$ denotes the exploration noise and $\mathrm{clip}(\cdot)$ ensures that all parameter values remain within their admissible intervals. The noise magnitude is gradually reduced during training so that the agent initially explores diverse parameter configurations and subsequently produces a more stable control policy.

2) Critic Network: The critic network $Q(\mathbf{s}_t, \mathbf{a}_t \mid \theta^Q)$ estimates the expected discounted return associated with applying action $\mathbf{a}_t$ in state $\mathbf{s}_t$. Unlike the actor, which receives only the state vector, the critic jointly processes the graph-aware state and the continuous ACO parameters:

$$Q_t = Q(\mathbf{s}_t, \mathbf{a}_t \mid \theta^Q). \tag{20}$$

Each observed transition

$$(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}) \tag{21}$$

is stored in an experience replay buffer. During learning, a mini-batch of transitions is sampled to reduce temporal correlation between consecutive observations.

The critic is optimized by minimizing the temporal-difference loss

$$\mathcal{L}_Q = \frac{1}{B}\sum_{i=1}^{B}\left(y_i - Q(\mathbf{s}_i, \mathbf{a}_i \mid \theta^Q)\right)^2, \tag{22}$$

where B denotes the mini-batch size and the target value is

$$y_i = r_i + \gamma Q'\left(\mathbf{s}_{i+1}, \mu'(\mathbf{s}_{i+1} \mid \theta^{\mu'}) \mid \theta^{Q'}\right). \tag{23}$$

Here, γ is the discount factor, while μ' and Q' denote the target actor and target critic, respectively.

The actor parameters are updated using the deterministic policy gradient

$$\nabla_{\theta^\mu} J \approx \frac{1}{B}\sum_{i=1}^{B} \nabla_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a} \mid \theta^Q)\Big|_{\substack{\mathbf{s}=\mathbf{s}_i \\ \mathbf{a}=\mu(\mathbf{s}_i)}} \nabla_{\theta^\mu}\mu(\mathbf{s}_i \mid \theta^\mu). \tag{24}$$

To stabilize learning, the target-network parameters are updated gradually using

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}, \tag{25}$$

$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}, \tag{26}$$

where $\tau \ll 1$ is the soft-update coefficient.

The use of a deterministic continuous policy allows the controller to produce smooth and fine-grained adjustments of all three ACO parameters. In contrast to a discrete controller that switches among predefined configurations, the proposed controller can generate parameter combinations specifically adapted to the current graph condition, pheromone distribution, convergence status, and stage of optimization.

G. Reward Function

The reward function provides the learning signal used to evaluate whether the parameter configuration selected by the controller improves the subsequent ACO search. Since individual ACO iterations may exhibit considerable stochastic variation, the reward is not computed after every iteration. Instead, one control cycle consists of $C = 5$ consecutive ACO iterations executed using the same parameter configuration. This interval allows the effect of an action to be observed over multiple solution-construction and pheromone-update steps.

Let L_t^{pre} denote the best tour length available immediately before control action $\mathbf{a}_t$ is applied, and let L_t^{post} denote the best tour length obtained after completing the corresponding control cycle. The immediate reward is defined according to the relative improvement in tour quality:

$$r_t = \frac{L_t^{\mathrm{pre}} - L_t^{\mathrm{post}}}{L_t^{\mathrm{pre}} + \varepsilon}, \tag{27}$$

where ε is a small positive constant introduced for numerical stability.

Under this definition, $r_t > 0$ indicates that the selected parameter configuration leads to a shorter tour, whereas $r_t = 0$ indicates that no improvement is obtained during the current control cycle. A negative reward may occur when the evaluated tour quality deteriorates under changing edge conditions. The relative formulation normalizes the improvement with respect to the current solution scale, thereby allowing reward values obtained from TSPLIB instances of different sizes and distance ranges to remain more comparable.

The reward is deliberately based on optimization improvement rather than on a manually prescribed target parameter configuration. Consequently, the agent is not instructed to prefer specific values of α, β, or ρ. Instead, it learns parameter combinations according to their observed effects on solution quality under different graph and search conditions.

Table III: Curriculum used to train the proposed DDPG controller.

Stage	Episodes	Available instances
1	1–100	`kroA200`
2	101–200	`kroA200`, `pr439`
3	201–300	`kroA200`, `pr439`, `rat575`
4	301–400	`kroA200`, `pr439`, `rat575`, `u1060`

H. Training Procedure

The proposed controller is trained through repeated closed-loop interactions with the dynamic ACO environment. At the beginning of each episode, a TSPLIB instance is selected according to the current curriculum stage. The dynamic graph environment and ACO optimizer are then initialized, and the initial 34-dimensional state vector is constructed from the graph descriptors and search statistics.

At each control step, the actor produces a continuous action corresponding to the three ACO parameters. Exploration noise is added during training, and the resulting values are mapped to their admissible parameter intervals. ACO then executes five iterations using the selected configuration. During these iterations, the dynamic environment updates the active edge-weight matrix, the ants construct candidate tours, and the pheromone matrix is updated.

After the control cycle, the reward is calculated from the relative improvement in the best tour length, and the next graph-aware state is extracted. The transition

$$(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}) \tag{28}$$

is stored in the replay buffer. Once a sufficient number of experiences has been collected, mini-batches are sampled to update the critic and actor networks using Eqs. (22) and (24), respectively. The target networks are subsequently updated using the soft-update rule in Eq. (26).

1) Curriculum Learning: Directly training the controller on large graph instances from the beginning may produce unstable rewards and make it difficult for the policy to identify meaningful relationships between the observed state and parameter actions. To alleviate this difficulty, the training process follows a four-stage curriculum in which increasingly large and challenging instances are introduced progressively.

During the first stage, the controller learns fundamental relationships between ACO search behavior and parameter adjustments on the relatively small `kroA200` instance. Larger instances are subsequently added without removing the previously introduced ones. This design preserves experience across graph scales while gradually exposing the policy to more complex optimization landscapes.

The `pcb1173` instance is excluded from all curriculum stages and is reserved for zero-shot evaluation. Its use during testing therefore examines whether the learned policy can transfer its parameter-control behavior to an unseen graph rather than merely reproduce configurations learned for the training instances.

Model checkpoints are saved every 50 episodes, allowing the training process to be resumed and enabling intermediate policies to be retained for subsequent evaluation.

Algorithm 1. Training Procedure of GARL-ACO

Require: Training instances organized according to Table III; number of episodes E; control interval $C = 5$

Ensure: Trained actor policy $\mu(\cdot \mid \theta^{\mu})$

Initialize actor $\mu(\cdot \mid \theta^{\mu})$ and critic $Q(\cdot, \cdot \mid \theta^{Q})$
Initialize target networks with $\theta^{\mu'} \leftarrow \theta^{\mu}$ and $\theta^{Q'} \leftarrow \theta^{Q}$
Initialize replay buffer $\mathcal{D}$
for $e = 1$ to E **do**
    Determine the curriculum stage associated with episode e
    Select a training instance from the instances available in the current stage
    Initialize the dynamic graph environment and ACO optimizer
    Initialize the ACO parameters and pheromone matrix
    Construct the initial graph-aware state $\mathbf{s}_0$
    Set $t \leftarrow 0$
    while the episode termination condition is not satisfied **do**
        Obtain a normalized action from the actor:

$$\widehat{\mathbf{a}}_t = \mu(\mathbf{s}_t \mid \theta^{\mu}) + \boldsymbol{\epsilon}_t$$

        Map and clip $\widehat{\mathbf{a}}_t$ to obtain $\mathbf{a}_t = (\alpha_t, \beta_t, \rho_t)$
        Record the pre-cycle best tour length L_t^{pre}
        for $c = 1$ to C **do**
            Update the active edge-weight matrix according to the dynamic perturbation model

Construct tours using the current values of $(\alpha_t, \beta_t, \rho_t)$
Evaluate the tours and update pheromone trails
Update the global best solution and ACO search statistics
end for
Record the post-cycle best tour length L_t^{post}
Compute r_t using Eq. (27)
Construct the next graph-aware state $\mathbf{s}_{t+1}$
Store $(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1})$ in $\mathcal{D}$
if the replay buffer contains sufficient experiences **then**
Sample a mini-batch from $\mathcal{D}$
Update the critic by minimizing Eq. (22)
Update the actor using Eq. (24)
Soft-update the target actor and critic using Eq. (26)
end if
$\mathbf{s}_t \leftarrow \mathbf{s}_{t+1}$
$t \leftarrow t + 1$
end while
Decay the exploration-noise magnitude
if e is divisible by 50 **then**
Save the current actor and critic checkpoints
end if
end for
return $\mu(\cdot \mid \theta^{\mu})$

III. Experimental Results

A. Experimental Setup

GARL-ACO was evaluated on five benchmark instances from the TSPLIB repository, namely `kroA200`, `pr439`, `rat575`, `u1060`, and `pcb1173`. Following the curriculum learning strategy described in Section III, the first four instances were employed for training, whereas `pcb1173` was reserved exclusively for zero-shot evaluation.

Dynamic routing scenarios were simulated by periodically perturbing 12% of graph edges while preserving the graph topology. The perturbation amplitude and period were fixed at 0.5 and 20 optimization iterations, respectively. Three optimization methods were compared:

- Fixed ACO with manually tuned parameters;
- ACO with Q-learning-based parameter control;
- GARL-ACO.

Table IV: Solution quality on dynamic TSPLIB instances.

Instance	Method	Mean Tour	Gap (%)
kroA200	Fixed ACO	64,336.9	119.07
	Q-learning	34,050.6	15.94
	GARL-ACO	**33,065.4**	**12.59**
pr439	Fixed ACO	571,182.4	432.73
	Q-learning	125,230.4	16.80
	GARL-ACO	**122,082.0**	**13.86**
rat575	Fixed ACO	8,118.8	19.87
	Q-learning	8,220.8	21.38
	GARL-ACO	**7,937.9**	**17.20**
u1060	Fixed ACO	1,875,850.0	737.08
	Q-learning	322,301.3	43.82
	GARL-ACO	**285,826.8**	**27.55**
pcb1173*	Fixed ACO	96,157.7	69.02
	Q-learning	78,289.2	37.61
	GARL-ACO	**71,139.2**	**25.04**

* Zero-shot evaluation.

Each method was independently executed three times using different random seeds, with 150 optimization iterations per run.

The performance was evaluated using four metrics: (i) mean tour length, (ii) optimality gap with respect to the best-known TSPLIB solution, (iii) standard deviation of the optimality gap, and (iv) recovery iterations after dynamic graph perturbations.

B. Overall Performance Comparison

Table IV compares the solution quality of the three optimization methods on all dynamic TSPLIB benchmark instances.

Table IV shows that GARL-ACO consistently achieves the best solution quality across all benchmark instances. Compared with conventional Fixed ACO, GARL-ACO substantially reduces both the mean tour length and the optimality gap under dynamic routing conditions. The performance improvement becomes increasingly significant as the problem size grows, indicating that adaptive parameter control is particularly beneficial for large-scale optimization problems.

Compared with the Q-learning controller, GARL-ACO further improves solution quality on every benchmark instance. Although both methods employ reinforcement learning to adapt ACO parameters, the continuous action space of the DDPG controller in GARL-ACO enables smoother and more precise parameter adjustment than the discrete action selection mechanism used by Q-learning.

The largest improvement is observed on the `u1060` instance, where the optimality gap decreases from 737.08% for Fixed ACO and 43.82%

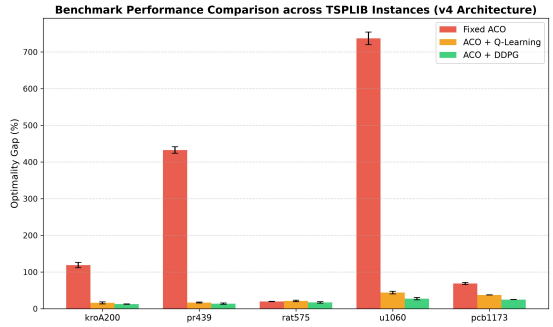


Figure 2: Comparison of the average optimality gap (%) achieved by Fixed ACO, Q-learning ACO, and GARL-ACO on five TSPLIB benchmark instances.

for Q-learning to only 27.55% with GARL-ACO. Similarly, on the previously unseen `pcb1173` instance, which is excluded from the curriculum learning process, GARL-ACO achieves the lowest optimality gap of 25.04%, compared with 37.61% for Q-learning and 69.02% for Fixed ACO. This result demonstrates that the learned GARL-ACO parameter-control policy generalizes well across different graph scales and unseen routing instances.

The results confirm that the combination of graph-aware state representation and continuous reinforcement learning in GARL-ACO enables more effective parameter adaptation than both manually tuned ACO and discrete RL-based parameter control, leading to consistently improved optimization performance in dynamic routing environments.

Figure 2 provides a visual comparison of the average optimality gap across all benchmark instances. Consistent with the numerical results reported in Table IV, GARL-ACO achieves the lowest optimality gap on every instance. The performance advantage becomes increasingly significant as the graph size increases, indicating that graph-aware continuous parameter control improves both optimization quality and scalability under dynamic routing conditions.

C. Adaptation and Recovery Analysis

Besides solution quality, the ability to rapidly adapt to environmental changes is equally important for dynamic routing problems. Table V summarizes the stability and recovery performance of the compared methods.

Table V demonstrates that GARL-ACO consistently exhibits stronger robustness under dynamic graph perturbations. The average recovery time is

Table V: Stability and recovery performance.

Instance	Method	Std Gap	Recovery
kroA200	Fixed ACO	7.17	1.0
	Q-learning	2.65	7.3
	GARL-ACO	**0.35**	**0.0**
pr439	Fixed ACO	8.87	1.0†
	Q-learning	**1.16**	4.3
	GARL-ACO	1.76	**0.7**
rat575	Fixed ACO	**0.44**	**0.0**
	Q-learning	1.94	7.3
	GARL-ACO	1.90	**0.0**
u1060	Fixed ACO	17.32	1.0‡
	Q-learning	3.52	1.0
	GARL-ACO	**3.16**	**0.0**
pcb1173*	Fixed ACO	3.06	1.0
	Q-learning	0.27	1.0
	GARL-ACO	**0.27**	**0.0**

* Zero-shot evaluation.
† One out of three runs failed to recover.
‡ Two out of three runs failed to recover.

reduced to zero iterations on four of the five benchmark instances, whereas Q-learning typically requires several additional optimization iterations before recovering. Although Fixed ACO occasionally achieves low variance on relatively small instances, its inability to adapt the parameter configuration results in poor recovery and significantly inferior solution quality on large-scale problems.

The improved adaptability of GARL-ACO can be attributed to its graph-aware state representation. Instead of relying solely on optimization statistics, GARL-ACO jointly exploits graph structural characteristics and search dynamics, allowing the controller to distinguish environmental changes from search stagnation and respond with appropriate parameter adjustments.

GARL-ACO converges substantially faster than both baseline methods while maintaining considerably smoother optimization trajectories after graph perturbations. In contrast, Fixed ACO exhibits pronounced oscillations because its static parameter configuration cannot respond to environmental changes. Although Q-learning partially alleviates this issue, its discrete action space limits the flexibility of parameter adaptation, leading to slower recovery after perturbations. These observations are consistent with the quantitative results reported in Tables IV and V.

D. Analysis of Learned Parameter Dynamics

Unlike conventional ACO and Q-learning, GARL-ACO does not converge to a fixed parameter configuration. Instead, its actor network continuously generates different parameter values accord-

ing to both the graph condition and the current search state.

Immediately after graph perturbations, the evaporation rate ρ temporarily increases, accelerating the removal of obsolete pheromone information and encouraging exploration of alternative routes. During relatively stable optimization stages, the pheromone influence factor α gradually increases, reinforcing high-quality paths that have consistently demonstrated good performance. Meanwhile, the heuristic influence factor β remains comparatively stable throughout the search, indicating that heuristic information provides reliable guidance regardless of environmental changes.

These adaptive behaviors reveal that the GARL-ACO controller learns distinct functional roles for the three ACO parameters rather than treating them independently. The coordinated adjustment of (α, β, ρ) enables GARL-ACO to maintain a more effective exploration–exploitation balance, thereby explaining the consistent improvements in solution quality, convergence speed, and recovery capability observed throughout the experimental evaluation.

IV. Conclusion

This paper presented GARL-ACO, a framework for online parameter control in Ant Colony Optimization for the Dynamic Traveling Salesman Problem (DTSP). Instead of relying on manually predefined parameter settings, GARL-ACO formulates parameter adaptation as a continuous reinforcement learning problem in which a Deep Deterministic Policy Gradient (DDPG) agent dynamically adjusts the three key ACO parameters, namely the pheromone influence factor α, heuristic influence factor β, and pheromone evaporation rate ρ. To improve environmental awareness, GARL-ACO employs a graph-aware state representation that integrates graph and environmental descriptors with optimization-related statistics, enabling the controller to jointly perceive changes in the routing environment and the internal search dynamics.

Experimental evaluation on dynamic TSPLIB benchmark instances demonstrated the effectiveness of GARL-ACO. Compared with both conventional fixed-parameter ACO and a Q-learning-based adaptive controller, GARL-ACO consistently achieved superior solution quality across all evaluated instances. Its advantage became increasingly evident on large-scale routing problems, where continuous parameter control enabled more effective adaptation to dynamic edge-cost perturbations. Furthermore, the learned GARL-ACO policy successfully generalized to the previously unseen `pcb1173` instance, indicating that its parameter-control behavior was not restricted to the graphs encountered during training.

The experimental results suggest that the performance improvement arises from the complementary roles of the two principal components of GARL-ACO. The graph-aware state representation enables the controller to distinguish changes caused by environmental perturbations from those resulting from search stagnation, while continuous action generation allows smooth and fine-grained adjustment of the ACO search behavior. Their integration within the GARL-ACO closed-loop learning process provides an effective mechanism for maintaining the exploration–exploitation balance under dynamically changing optimization environments.

Although GARL-ACO demonstrates promising performance, several limitations remain. The current study considers dynamic edge-weight perturbations while preserving graph topology, and the reinforcement learning policy is developed for a single optimization algorithm. Future work will investigate more complex dynamic routing scenarios involving structural graph changes, extend the GARL-ACO parameter-control strategy to other swarm intelligence and evolutionary optimization algorithms, and explore graph neural network representations for learning richer environment-aware state embeddings. Another promising direction is to evaluate GARL-ACO on real-world transportation datasets to further assess its scalability and practical applicability.

References

[1] V. Pillac, M. Gendreau, C. Guéret, and A. L. Medaglia, "A review of dynamic vehicle routing problems," *European Journal of Operational Research*, vol. 225, no. 1, pp. 1–11, 2013.

[2] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Cambridge, MA: MIT Press, 2004.

[3] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996.

[4] M. Dorigo and L. M. Gambardella, "Ant colony system: A cooperative learning approach to the traveling salesman problem," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 53–66, 1997.

[5] C. Blum, "Ant colony optimization: Introduction and recent trends," *Physics of Life Reviews*, vol. 2, no. 4, pp. 353–373, 2005.

[6] R. S. Sutton and A. G. Barto, "Reinforcement learning: An introduction," 2018.

[7] V. Mnih, K. Kavukcuoglu, D. Silver *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[8] E. B. Khalil, H. Dai, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[9] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization: A methodological tour d'horizon," *European Journal of Operational Research*, vol. 290, no. 2, pp. 405–421, 2021.

[10] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *International Conference on Learning Representations (ICLR)*, 2016.